TPL Trakker AI_APP Documentation

1. Overview

This application performs real-time human detection, face detection, and gender classification using YOLOv8 and a ViT (Vision Transformer) model. It logs analytics such as the number of people, gaze detection (looking at the camera), and gender distribution. Data is stored in a CSV file with hourly summaries. The app includes a GUI built with Tkinter and requires login once every 30 days.

2. Features

- Human detection using YOLOv8

- Gender classification using ViT model from Hugging Face

- Face detection using OpenCV Haar Cascade

- Real-time video logging

- Hourly and termination summaries

- GUI with Start/Stop detection buttons

- Login prompt once every 30 days

3. Technologies Used

- Python 3

- OpenCV

- Ultralytics YOLOv8

- Hugging Face Transformers (ViT model)

- Tkinter for GUI

- PyTorch with CUDA (if available)

4. Functionality Breakdown

4.1 Human Detection

Uses YOLOv8 model to detect humans in video frames. Returns bounding boxes for persons with confidence > 0.5.

4.2 Gender Classification

Applies the Hugging Face ViT model to cropped face images to classify them as male or female.

4.3 Logging System

Logs are written to a CSV file. Hourly summaries and final termination summaries are added to analyze trends over time.

4.4 GUI

The GUI contains buttons to start and stop detection. Detection runs in a separate thread to keep the UI responsive.

4.5 Login System

Checks a `last_login.txt` file to see if login is required. If the last login was over 30 days ago or the file doesn't exist, it prompts for login.

5. Output Files

- Video recordings stored as `.avi` files under the `camera/` directory

- CSV log files with timestamped detection data and summaries

6. Dependencies

- ultralytics

- torch

- transformers

- opencv-python

- tkinter (built-in)

7. Installation and Executable Creation

7.1 Installation Steps

1. Create a virtual environment (optional but recommended):

```bash
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

2. Install the required Python packages:

```bash
pip install ultralytics torch torchvision torchaudio
pip install transformers opencv-python
```

3. Ensure that CUDA is properly installed if you want to leverage GPU acceleration.

7.2 Creating a Standalone Executable (EXE)

To bundle the application into a standalone `.exe` file using PyInstaller:

1. Install PyInstaller:

```bash
pip install pyinstaller
```

2. Create the executable (run this in the directory containing your script):

```bash
pyinstaller --onefile --noconsole --add-data "haarcascade_frontalface_default.xml;." --add-data "last_login.txt;." your_script.py
```

- `--onefile`: Bundle into one `.exe` file
- `--noconsole`: Hide the terminal window for GUI apps
- `--add-data`: Include required data files like Haar cascade and `last_login.txt`
- Use a semicolon (;) as the separator on Windows, and a colon (:) on Unix-based systems.

3. The final executable will be located inside the `dist/` folder.

7.3 Note on CUDA & Torch with Executable

If you're using CUDA with PyTorch, ensure the machine running the `.exe` has the appropriate CUDA drivers installed. Torch with CUDA doesn't bundle the CUDA runtime into the executable.

To use the application:

1. Run the application (either the Python script or the generated `.exe` file).

2. A login window will appear. Enter the credentials:

- Username: `------`
- Password: `--------`

3. After successful login, the main GUI window will open.

4. Click `Start Detection` to begin analyzing human presence and gaze.

5. Click `Stop Detection` to stop the analysis and save a final summary.

6. Close the application using the GUI or the close (X) button on the window